

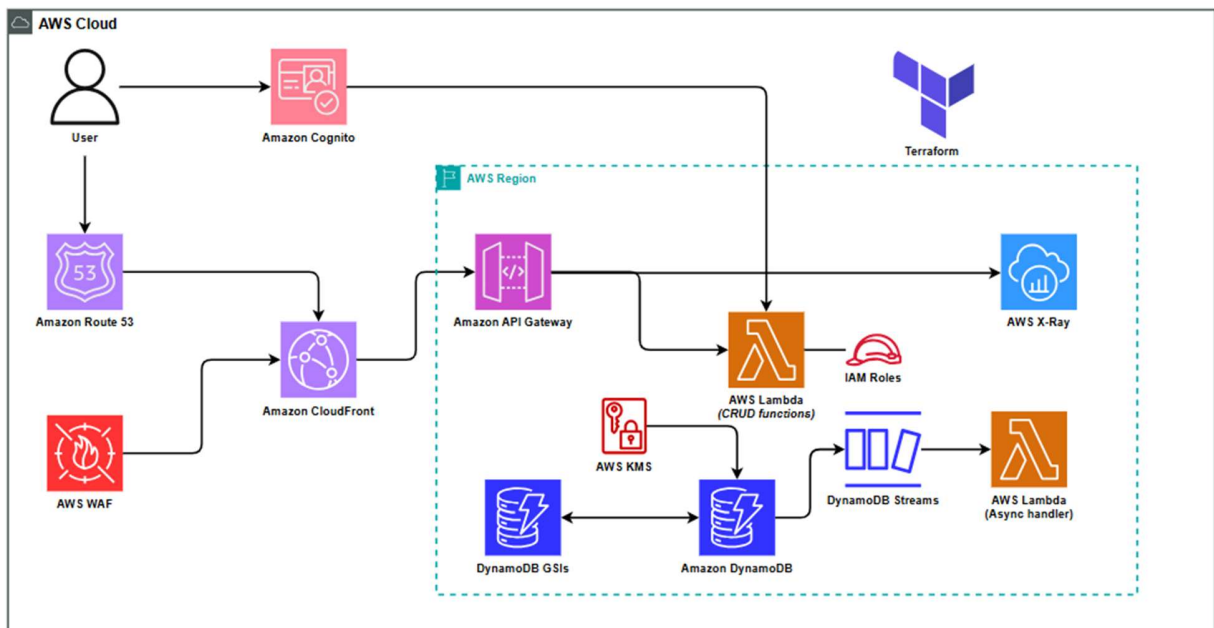
AWS Serverless REST API

Project Overview

This project implements a scalable, secure, and serverless REST API for task management using AWS managed services. The solution is designed to provide high availability, low operational overhead, and seamless scalability while following AWS Well-Architected Framework principles.

The implementation was deployed and tested using the AWS Builder Center Sandbox environment to validate the architecture under real-world conditions.

Architecture Diagram



1. Architecture Design Philosophy

The application follows a fully serverless architecture to eliminate infrastructure management and enable automatic scaling. Compute, storage,

authentication, and edge services are strategically decoupled to improve maintainability, resiliency, and performance.

The architecture is designed around the following AWS services:

- Amazon CloudFront
- AWS WAF
- Amazon API Gateway
- Amazon Cognito
- AWS Lambda
- Amazon DynamoDB
- AWS X-Ray
- Amazon CloudWatch
- Terraform (Infrastructure as Code)

2. Key AWS Services & Integration

Amazon API Gateway

Handles REST endpoint management, request routing, and API caching to reduce backend load.

AWS Lambda

Executes event-driven CRUD operations securely via IAM roles and environment variables.

Amazon DynamoDB

Provides NoSQL data persistence with a robust table design and Global Secondary Indexes (GSIs) for low-latency access patterns.

Amazon Cognito

Manages centralized user authentication and authorization using JWT tokens.

AWS WAF

Protects the API against common web exploits, rate-based attacks, and bots.

Amazon CloudFront

Delivers global edge caching and acts as a CDN in front of the API Gateway for optimized delivery.

AWS X-Ray

Enables end-to-end distributed tracing across the serverless call chain for performance analysis and debugging.

Amazon CloudWatch

Ensures comprehensive logging, monitoring, and operational visibility.

Terraform

Provides Infrastructure as Code (IaC) capabilities for provisioning, managing, and version-controlling AWS infrastructure resources consistently across environments.

3. Security & Operational Excellence

Identity Management

Amazon Cognito is fully integrated to enforce secure, token-based authentication (JWT) at the API Gateway level.

Threat Protection

AWS WAF is actively configured with managed OWASP rule sets to protect API endpoints from malicious injections, abusive traffic patterns, and automated bot attacks.

Observability & Debugging

AWS X-Ray traces requests across API Gateway, Lambda, and DynamoDB, identifying latency bottlenecks and ensuring operational health throughout the application lifecycle.

Monitoring

Amazon CloudWatch captures all application logs, operational metrics, alarms, and performance indicators to maintain visibility across the entire serverless stack.

4. Implementation Highlights

Core Features Implemented

- Create Tasks
- Retrieve Tasks
- Delete Tasks

REST Endpoints

Method	Endpoint
POST	/tasks
GET	/tasks
DELETE	/tasks/{id}

Serverless Benefits Achieved

- Zero server provisioning or management overhead.
- Automatic scaling to meet demand spikes.
- Cost-optimized, pay-per-use architecture.
- High availability and fault tolerance by design.
- Improved security through managed AWS services.
- Reduced operational complexity and maintenance effort.

5. Infrastructure as Code (IaC)

Terraform was utilized to provision and manage AWS infrastructure components in a consistent, repeatable, and automated manner.

Key benefits of using Terraform include:

- Declarative infrastructure management.

- Repeatable deployments.
- Improved infrastructure consistency.
- Enhanced maintainability and scalability.
- Version-controlled infrastructure definitions.

6. Future Enhancements

While the current architecture is robust, it can be further extended by:

- Integrating DynamoDB Streams for downstream event processing and real-time analytics.
- Hosting a static frontend application (React/Vue) directly on Amazon S3 and serving it globally through CloudFront.
- Implementing fully automated CI/CD pipelines using AWS CodePipeline and AWS CodeBuild.
- Adding Amazon EventBridge for event orchestration.
- Integrating Amazon SNS for notification delivery.
- Expanding Terraform modules for multi-environment deployments.

7. Conclusion

This project demonstrates a production-ready application of AWS serverless services to build a highly scalable, secure, and maintainable backend with minimal operational effort.

By leveraging AWS managed services and Infrastructure as Code (Terraform), the solution achieves enterprise-grade scalability, security, observability, and fault tolerance while maintaining a modern serverless architecture capable of supporting complex business logic and future business growth.